Dashboard host-render — screen×STATE gap closure (EMPTY + ERROR variants)

Revision: 1 **Last modified:** 2026-07-10T00:20:00Z **Scope:** `dashboard/hostrender/` (host-render harness + Playwright specs) — no `clients/ota-manager/`, `server/`, or `design-systems/` files touched. **Authority:** Helix Constitution §11.4.170 (device-independent host-rendered UI visual proof per screen×STATE×{light,dark}), §11.4.107(10) (self-validated golden-good/golden-bad analyzer), §11.4.6 (no-guessing — no fabricated UI state), §11.4.3 (honest SKIP-with-reason where a state genuinely does not exist in the component).

---

1. Problem statement

The existing `dashboard/hostrender/screens.hostrender.spec.ts` host-renders 8 screens (+ the standalone `login.hostrender.spec.ts`) but only in their **DEFAULT (populated-data)** state, × {light,dark}. §11.4.170 requires proof per screen×**STATE**×theme, not just per screen×theme with state fixed at "happy path". This round closes that gap for 2 core screens whose empty-list and error-panel branches are user-visible and already implemented in source:

- **ReleaseList** (`src/screens/ReleasesScreen.tsx`) — real `EmptyState` ("No releases yet.") and real `ErrorPanel` branches, gated on `data.items.length === 0` / `error` respectively.
- **FleetHealth** (`src/screens/FleetScreen.tsx`) — real `EmptyState` ("No device states reported yet." / "No telemetry events yet.") and real `ErrorPanel` branches on the `/telemetry/overview` aggregate.

No fabricated state was invented: both branches were located in the existing, already-shipped component source (see §2 below) before any test was written, per §11.4.6 no-guessing.

2. How the states are induced (real code path, not a synthetic DOM mutation)

`dashboard/hostrender/harness-main.tsx` gained an optional `&state=empty|error` query param (default `default`, fully backward-compatible — every other screen ignores it and renders exactly as before). It is consulted ONLY for the two endpoints these two screens depend on:

Screen	Endpoint	`state=empty` stub	`state=error` stub
ReleaseList	`GET /releases`	`{ items: [] }` (200)	`{ error: { code: "INTERNAL", message: "Releases backend unavailable." } }` (500)
FleetHealth	`GET /telemetry/overview`	`{ event_counts: {}, total: 0, failure_rate: 0, by_state: {} }` (200)	`{ error: { code: "INTERNAL", message: "Telemetry overview backend unavailable." } }` (500)

500 (not 404) was used deliberately for the ERROR variant: FleetHealth's own source special-cases a 404 on `/telemetry/overview` into a DIFFERENT "endpoint not available" `EmptyState` (a graceful-degrade branch, not the `ErrorPanel` branch) — so a 404 would have exercised a THIRD state, not the intended error-panel state. A genuine 500 reaches the real `ErrorPanel` branch for both screens (`error instanceof ApiError` in `ReleaseList`; `overview.error && !overviewMissing` in `FleetHealth`).

This is a component-render stub at the network boundary only (permitted per §11.4.27) — every pixel sampled is the real `ReleaseList` / `FleetHealth` component rendering its own real conditional branch against real response data, exactly as production code would on an empty backend or a 500.

`dashboard/hostrender/screens.hostrender.spec.ts` gained a `screenParam` / `stateParam` field on `ScreenSpec` (both optional, default to the existing behavior) and 4 new specs (`releases-empty`, `releases-

`error`, `fleet-empty`, `fleet-error`) that reuse the **IDENTICAL** generic test-generation loop as the 8 existing screens — same dual-oracle discipline, zero duplicated test logic:

1. golden-good `toHaveScreenshot` against a committed baseline,
2. DOM-bounds + rendered-text layout oracle, self-validated (PASS on the real render, FAIL-detected on an injected regression),
3. explicit pixelmatch image-diff analyzer, self-validated (~0 good↔good, large good↔mutated),
4. committed golden baseline provably REJECTS a mutated render,
5. light↔dark distinctness (dark is a genuine re-theme, not a recolor no-op).

3. Verdict table — screen × STATE × theme

Screen	State	Theme	golden-good	layout oracle (self-val)	image-diff (self-val)	baseline-rejects-mutated	light↔ distinc
ReleaseList	EMPTY	light	PASS	PASS	PASS (good↔good 0.0%, good↔mutated 51.2%)	PASS (diffRatio 0.51)	—
ReleaseList	EMPTY	dark	PASS	PASS	PASS	PASS	—
ReleaseList	EMPTY	—	—	—	—	—	PASS (r 0.975)
ReleaseList	ERROR	light	PASS	PASS	PASS	PASS	—
ReleaseList	ERROR	dark	PASS	PASS	PASS	PASS	—
ReleaseList	ERROR	—	—	—	—	—	PASS
FleetHealth	EMPTY	light	PASS	PASS	PASS	PASS	—
FleetHealth	EMPTY	dark	PASS	PASS	PASS	PASS	—
FleetHealth	EMPTY	—	—	—	—	—	PASS
FleetHealth	ERROR	light	PASS	PASS	PASS	PASS (diffRatio 0.514)	—
FleetHealth	ERROR	dark	PASS	PASS	PASS	PASS (diffRatio 0.517)	—

Screen	State	Theme	golden-good	layout oracle (self-val)	image-diff (self-val)	baseline-rejects-mutated	light ↔ distinc
FleetHealth	ERROR	—	—	—	—	—	PASS

All 36 new tests (4 states × 9 tests/state: 4 test-types × 2 themes + 1 theme-distinctness) PASS on a clean run (no `--update-snapshots`). Full captured-evidence artefacts (JSON verdicts + PNG actual/diff frames) for every cell are under `docs/qa/20260709-dashboard-state-variations/` — see §5.

Sample self-validation readouts (proving the analyzers are not tautologies, §11.4.107(10)):

- `image-diff-selfcheck-releases-empty-light.json`: `goodVsGood.ratio = 0`, `goodVsMutated.ratio = 0.5123`, verdict `SELF-VALIDATED`.
- `baseline-rejects-mutated-fleet-error-dark.json`: `diffRatio = 0.5171`, verdict `GOLDEN-BAD: committed baseline rejects the mutated render`.
- `light-vs-dark-distinctness-releases-empty.json`: `ratio = 0.9749`, verdict `DISTINCT: dark is a genuine re-themed surface`.

4. SKIP / honest-boundary notes (§11.4.3, §11.4.6)

None. Both target screens (`ReleaseList`, `FleetHealth`) were confirmed, by reading `src/screens/ReleasesScreen.tsx` and `src/screens/FleetScreen.tsx` BEFORE writing any test, to already implement a real, distinct EmptyState branch and a real, distinct ErrorPanel branch — no state needed to be invented, and no screen was skipped. `DeploymentList` (the task's third suggested option) was NOT used since 2 screens fully satisfied the task scope and reusing the identical generic loop for a 3rd would not have added discipline diversity; it remains a candidate for a future round if the operator wants broader screen×STATE coverage.

5. Before → after e2e counts (`playwright test --config=playwright.hostrender.config.ts`)

	Test files	Total tests	Result
Before (baseline, this stream's first action)	2 (`login.hostrender.spec.ts`, `screens.hostrender.spec.ts`)	81	81 passed
After (this round's additions)	2	117	117 passed
Δ	—	**+36**	all new tests green on a clean (non-update) run

`npm run build` → exit 0 (tsc ×2 + vite build, clean, no warnings).
`npm run test:run` (vitest unit suite) → **107/107 passed**, unchanged (no unit-test-scoped source was touched by this round — the change is additive, harness-only).

6. Files touched / added

- `dashboard/hostrender/harness-main.tsx` — added `&state=empty|error` param handling for `/releases` and `/telemetry/overview` only; added `TELEMETRY_OVERVIEW_EMPTY` + `RELEASES_EMPTY` canned payloads.
- `dashboard/hostrender/screens.hostrender.spec.ts` — added `EVIDENCE_DIR_STATES`, `screenParam`/`stateParam` on `ScreenSpec`, updated `gotoScreen` to build the URL from those fields, added 4 new `ScreenSpec` entries (`releases-empty`, `releases-error`, `fleet-empty`, `fleet-error`), updated `beforeAll` to `mkdirSync` the new evidence dir.
- `dashboard/hostrender/screens.hostrender.spec.ts-snapshots/` — 8 new committed golden PNGs: `releases-empty-{light,dark}-chromium-linux.png`, `releases-error-{light,dark}-chromium-linux.png`, `fleet-empty-{light,dark}-chromium-linux.png`, `fleet-error-{light,dark}-chromium-linux.png`.

- `docs/qa/20260709-dashboard-state-variations/` — this file + all captured per-test evidence (JSON verdicts, `*-actual.png`, `diff-bad-*.png`).

No files under `clients/ota-manager/`, `server/`, or `design-systems/` were read, referenced, or modified. No `git` command was run by this stream.